

SeekJudge: A Practical Reward Framework for Reinforcement Learning in Computer-Use Agents

Yang Wan¹, Zhenhao Zhang², Jierui Wang¹, Linchao Zhu^{*1}

¹Zhejiang University ²University of Rochester

*Corresponding author

Code: <https://github.com/ZJUSCL/SeekJudge>

Model: <https://huggingface.co/ZJUSCL/SeekJudge-9B>

Benchmark: <https://huggingface.co/datasets/ZJUSCL/CUASStepBench>

Abstract

Deciding whether a trajectory actually fulfills its instruction governs how we measure computer-use agents on long-horizon graphical-user-interface tasks and how we train them with reinforcement learning. This judgment has long relied on rule-based evaluation, which struggles to align with human intention and goes stale when an app updates or its online content drifts. Existing model-based judges attempt to address these problems but still leave a performance gap to the rule-based evaluation. We propose the **SEEKJUDGE** framework, in which four role-specialized agents, a Condense, a Ground, a Seek and an Analyze agent, reach a verdict through a Seek-Analyze loop over the trajectory. A seed-calibrated distillation pipeline trains one specialized 9B model to serve as the shared backbone for all four agents. Measured by downstream success rate on held-out RL test goals, SEEKJUDGE is the first practical model-based reward to match or surpass native rule-based supervision in online RL. Beyond accuracy, SEEKJUDGE provides step-level judgments, runs far cheaper than a closed-source large model, and keeps a small per-call context that scales to much longer trajectories. We further contribute a general architectural improvement to the reward server that speeds up judging in RL. Together these make model-based reward a practical drop-in for rule-based supervision in CUA reinforcement learning.

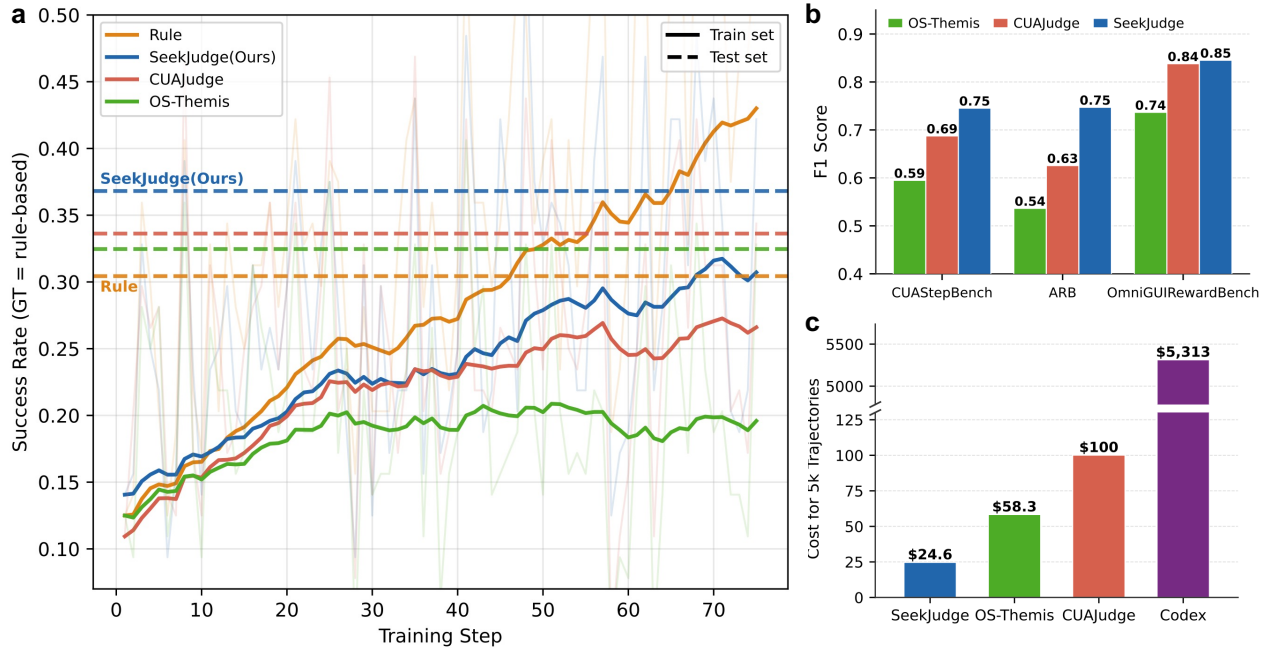


Figure 1 (a) RL success rate under each reward (dashed = test set) for UI-TARS 1.5 7B on Impress. (b) Offline F_1 on three benchmarks. (c) Cost to judge 5K trajectories, roughly the volume consumed by a single RL run in this work.

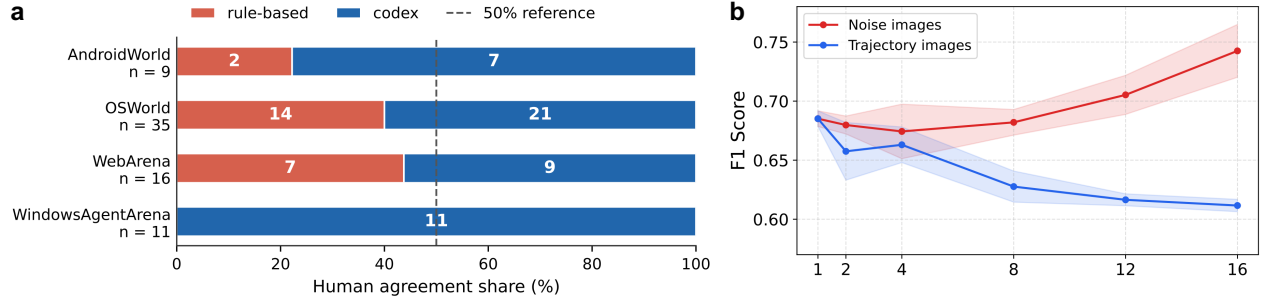


Figure 2 Two limitations that motivate SEEKJUDGE. **(a)** On rule-judge disagreement cases, the model judge (*blue*) agrees with human verdicts more often than the rule (*orange*) on all four benchmarks; from about 1k evaluated trajectories, we keep only the cases where the rule-based and model verdicts differ. **(b)** With the decisive screenshot always present, padding with other trajectory images (*blue*) lowers F_1 , while equal-token noise images (*red*) do not, implicating competing content rather than context length (Section 4.5).

1 Introduction

Computer-use agents perform long-horizon tasks over graphical user interfaces, and the field has advanced rapidly in the past few years. Early work targeted constrained synthetic environments such as MiniWoB (Shi et al., 2017), and the goal has since broadened toward open-ended tasks across web, mobile and desktop platforms (Zhou et al., 2024; Deng et al., 2023; Xie et al., 2024; Rawles et al., 2025). At the same time the dominant observation modality has shifted from accessibility trees (Zhou et al., 2024) toward general, visible GUI screenshots, as exemplified by native vision-based agents (Koh et al., 2024; Qin et al., 2025). Given a human instruction and the long trajectory an actor produces, deciding whether that trajectory actually fulfills the instruction is a central problem. The decision determines how we measure a model, and it governs the effectiveness of downstream reinforcement learning or rejection fine-tuning.

For a long time, judging a trajectory has relied mainly on rule-based evaluation (Xie et al., 2024; Rawles et al., 2025; Zhou et al., 2024; Wang et al., 2026), which suffers from three problems. **(1)** A rule struggles to align with human intention, as shown in Figure 2(a). Many self-consistent answers can satisfy the same goal, yet a rule cannot enumerate them all (Zhu et al., 2025). The extreme case is an open-ended instruction such as drawing a picture, where acceptable outputs are unbounded and no rule can judge the result. **(2)** It covers few environments and breaks when they change. A rule reads internal state through layout or internal APIs, so each new application must be bound to a dedicated parser or a purpose-built mock app. This confines a rule to a narrow set of applications at a frozen version. **(3)** Its ground-truth answer goes stale. A rule relies on a predefined answer, but once the underlying online content changes that answer drifts, or the task itself becomes unsolvable. Rule-based evaluation is therefore mostly limited to offline software, and covering live online services incurs a heavy maintenance cost (Xue et al., 2025).

These problems motivate a shift from rule-based to model-based evaluation, but existing frameworks (Xue et al., 2026; Li et al., 2026b; Rosset et al., 2026) have not yet brought this paradigm into the mainstream. Their central weakness is that they dilute the decisive evidence. They pour many images and observations into one context, while only a small high-fidelity fragment determines the verdict, so the model may hallucinate on minor details or over-trust the trajectory.

To address these problems holistically, we revisit what it takes to judge a long trajectory and frame it as the composition of two subtasks: localization, which identifies the image holding the decisive evidence, and extraction, which reads the decisive detail from that image accurately. We observe that, even when the decisive image is always provided, adding more images from the same trajectory to a single forward pass degrades performance increasingly with their number, as shown in Figure 2(b).

Motivated by this finding, we propose SEEKJUDGE, a model-based evaluation framework. Unlike common frameworks that perform localization and extraction together in one multi-image forward pass (Xue et al., 2026), SEEKJUDGE uses state summaries for localization that select an image id, and an analyze agent for high-fidelity extraction that reads one image at a time.

Existing model-based judges are validated chiefly by offline agreement with human or teacher-model verdicts, whereas serving as a practical reward model inside an RL loop requires jointly balancing fine-grained scoring, cost and latency. We therefore optimize SEEKJUDGE along quality and efficiency, so that it can serve as a practical reward model inside reinforcement learning.

- **On quality**, the framework outputs a fine-grained signal that scores the quality of every action in a trajectory, giving step-level judgments. We train a strong specialized 9B model for it. To support step-level evaluation we build CUASTEPBENCH and CUASTEPBENCH-LONG through dense human annotation. As Figure 1 shows, measured by downstream test success across applications, SEEKJUDGE is the first model-based reward to match or even surpass native rule-based supervision. The decoupled design keeps the per-call context small, so SEEKJUDGE extends to much longer trajectories that a single-pass judge cannot fit.
- **On efficiency**, we restrict the system to a single 9B open-source model, hundreds of times cheaper than a closed-source large model as Figure 1(c) shows, and we further propose asynchronous reward-model prefetch evaluation that runs part of the judging workload while the environment executes actions, cutting the time the reward model blocks the rollout. Together these make online reinforcement learning far more practical.

In summary, our contributions are as follows. **(1)** We propose SEEKJUDGE, an multi-round reward-model framework, designed to jointly address cost, step-level judgments and scalability, with engineering optimizations such as asynchronous prefetch evaluation that make it faster inside reinforcement learning. **(2)** We construct CUASTEPBENCH, the first CUA reward benchmark to pair human trajectory verdicts with dense step-level labels on the same executed trajectories, spanning 177 applications across platforms. **(3)** We train a specialized 9B model for the framework that matches or surpasses rule-based evaluation on offline reward benchmarks, advancing reward-model-driven reinforcement learning for computer-use agents.

2 Related Work

2.1 The Judging Problem in Computer-Use Agents

Computer-use agents (CUAs) operate graphical user interfaces by emitting keyboard and mouse actions and observing streams of screenshots, and progress has been paced by benchmarks. The field evolved from synthetic web control (Shi et al., 2017) through large-scale web instruction following (Deng et al., 2023) and functional success checks over the DOM (Zhou et al., 2024) or rendered screenshots (Koh et al., 2024), then moved off the browser to real desktop (Xie et al., 2024) and mobile (Rawles et al., 2025) environments, to native vision agents (Qin et al., 2025), and most recently to long-horizon workflows spanning many applications (Yuan et al., 2026). All of these settings presuppose one capability, deciding whether a trajectory actually fulfilled the instruction. Reliable rule checkers double as the reward for reinforcement learning (RL), and where they are absent RL work must build its own model-based evaluation (Bai et al., 2026). As trajectory data increasingly governs agent capability (Li et al., 2024), affordable and scalable judging has become part of the training loop rather than an evaluation afterthought.

2.2 From Rule-Based to Model-Based Evaluation

Rule-based verifiers dominate current benchmarks (Xie et al., 2024; Rawles et al., 2025; Zhou et al., 2024) and verifiable-training pipelines (Wang et al., 2026), but suffer three structural problems. They misalign with human intent, since many tasks admit correct answers no finite rule set can enumerate (Zhu et al., 2025). They inspect internal state through version-frozen parsers, a fragility that forced OSWorld to repair its evaluators after community-reported errors (Xie et al., 2025). Their predefined answers go stale as online content drifts; Xue et al. (2025) find a simple Google-Search agent already solves up to 51% of tasks on prior web benchmarks, so much reported progress reflects leaked or outdated checkers.

Model-based judges answer these pressures, yet existing frameworks leave gaps. CUAJudge (Xue et al., 2026) identifies key points and key screenshots before judging, but still scores whole trajectories within multi-image forward passes, where the decisive fragment is diluted among near-static frames and the judge over-trusts the

agent’s self-reports. OS-Themis (Li et al., 2026b) audits milestone evidence chains with a multi-agent critic, at the cost of large closed models and heavy context. UI-TARS-2 (Wang et al., 2025a) reuses the policy as its own outcome reward model, exposing an accuracy-versus-cost tension at scale. That verifiable-training work still prefers functional checks (Wang et al., 2026) shows model judges have not reached mainstream adoption, precisely the gap SEEKJUDGE targets.

Four recent works bear most directly on our central claim, and each stops short of it. WebJudge (Xue et al., 2025) narrows the human-agreement gap below that of rules, but only as an offline evaluator, never as an RL reward compared head-to-head with rule-based supervision. OpenWebRL-Judge (Yang et al., 2026) matches GPT-4.1 supervision in online web-agent RL with an open 8B judge, yet its control arm is another model judge, and it stays web-only and trajectory-level. OS-Themis (Li et al., 2026b) reports a 10.3-point RL gain, but not over rule-based supervision. PRO-CUA (He et al., 2026) reports PRM rewards beating a rule baseline in step-level web RL, but that rule matches golden reference actions rather than checking executed state, and success is itself scored by a GPT-5 judge, so its rule-versus-model comparison stays circular. None holds out the environment’s native rule verifiers as ground truth in online RL, the precise question SEEKJUDGE answers.

2.3 Step-Level Reward Modeling

The case for step-level supervision sharpens as trajectories lengthen. A trajectory-level verdict carries vanishing signal density and intractable credit assignment once horizons reach hundreds of steps, now the norm, with tasks often exceeding 500 steps (Aggarwal et al., 2026) and OSWorld 2.0 workflows averaging 318 tool calls versus about 30 in OSWorld 1.0 (Yuan et al., 2026). This echoes process supervision in mathematical reasoning, where step-level feedback outperforms outcome-only signals (Lightman et al., 2024; Wang et al., 2024).

Existing step-level judges for CUAs are each incomplete. SEAgent (Sun et al., 2025) localizes step errors but only on narrow Chrome tasks with poor generalization, showing naive specialized fine-tuning is insufficient. GUI-Owl (Ye et al., 2025) embeds step-level critics inside the actor pipeline rather than as an independent judge, without addressing multi-image dilution or cost. OpenCUA (Wang et al., 2025b) reflects per step but sees only context truncated to the current step. VLM-harvested rewards also drive GUI and web-agent RL (Yang et al., 2025; Qi et al., 2025; Bai et al., 2024) and trajectory filtering (Pan et al., 2024; He et al., 2024), but these are generic LLM-as-a-judge instances (Zheng et al., 2023) rather than dedicated step-level, cross-platform reward models.

Judge benchmarks reveal the same gap, in that step-level granularity and cross-platform coverage have never coexisted. AgentRewardBench (Lù et al., 2025) is trajectory-level and web-only. OmniGUIRewardBench (Li et al., 2026a) broadens platform coverage but stays tied to outcome rewards. CUARewardBench (Lin et al., 2025), the most directly overlapping parallel work, provides human step-level labels but annotates sparse key actions on a single Ubuntu platform. CUAVerifierBench (Rosset et al., 2026) offers only coarse per-step progress descriptions and covers only the web. OS-Critic Bench (Wu et al., 2026) comes closest on coverage, with human-labeled steps spanning desktop and Android, yet its 738 isolated steps ask a pre-execution question, namely whether a sampled candidate action would advance the task, and it carries no trajectory-level verdict. CUASTEPBENCH is, to our knowledge, the first CUA reward benchmark to pair human trajectory-level verdicts with dense post-hoc step labels on the same executed trajectories, and the first whose coverage extends to iOS, with SEEKJUDGE as the companion single-model system that turns dense step-level judgments into a practical RL reward.

3 Method

3.1 The SEEKJUDGE Framework

Judging a long trajectory requires evidence from many screenshots, but feeding them all into one forward pass degrades a VLM, as Figure 2 shows. A full trajectory also exceeds a single context window. Following the decomposition in Section 1, SEEKJUDGE splits judging into localization, which finds the step and image that hold the decisive evidence, and extraction, which reads that evidence at high fidelity. SEEKJUDGE realizes this decomposition as a multi-agent framework whose four agents share one backbone model, as Figure 3 shows. The Condense and Ground agents first compress the whole trajectory into a compact timeline. The

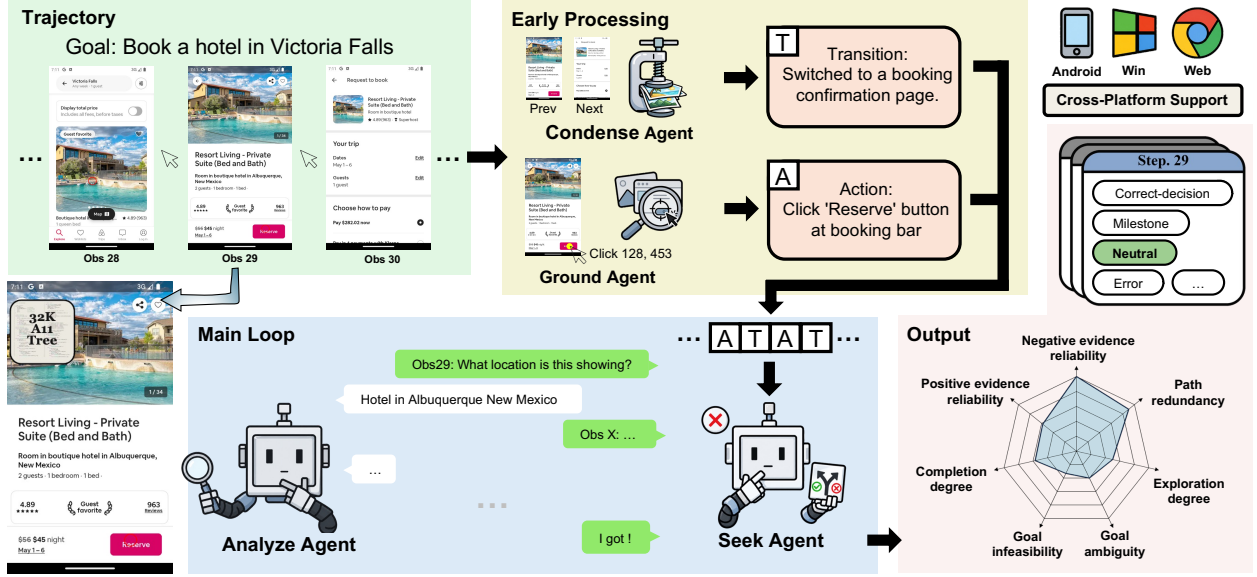


Figure 3 The SEEKJUDGE framework, where four agents share one backbone model.

Seek agent takes this timeline as its initial context and runs a Seek-Analyze loop, querying the Analyze agent over several rounds for the evidence it still needs before emitting the final judgment.

Seek agent. The Seek agent is the controller and the only agent that carries state across a run. It reads a condensed timeline that interleaves, for each step, a transition entry (T) from the Condense agent and a grounded-action entry (A) from the Ground agent, and it judges whether the accumulated evidence settles the instruction. When the evidence is enough, it emits the verdict, seven trajectory-level dimension scores together with a nine-way class label for each step. When it is not, it names a single step and the image id to inspect, sends a focused question to the Analyze agent, and appends the returned text to its context before the next round.

Analyze agent. The Analyze agent answers one focused question about the single screenshot the Seek agent names and returns the decisive detail as text. It is stateless: every call starts fresh with only that question and image, so it acts as a tool the Seek agent invokes for a detail, while the Seek agent alone tracks the run.

Condense agent. For each step the Condense agent reads the two screenshots before and after the action and writes a few lines describing the state transition, the T entry of the timeline. Run over consecutive pairs, it turns the whole trajectory into a compact text timeline before the loop begins.

Ground agent. For each step the Ground agent reads the post-action screenshot together with the raw action the actor executed, such as click 128,453, and identifies from the screenshot which element those coordinates actually hit, the A entry of the timeline.

Properties. Three properties follow from this decomposition. **(1)** Judgments stay robust to distraction. Each call sees few images yet reads them at high fidelity, minimizing interference from other images and observations. Section 4.5 quantifies this effect. **(2)** Evaluation scales to far longer trajectories. Decoupling localization from extraction keeps each call on a much smaller context, 4–6× smaller than existing reward-model methods. The Analyze agent in particular reads a screen on demand as a single image and returns only a short answer, so an observation whose raw accessibility tree can reach 32k tokens never enters the judging context. This raises the step limit a judge can handle and lowers training resource needed on long trajectories, as Section 4.7 reports. **(3)** Compute adapts to task difficulty. The Seek agent issues only as many extraction calls as a case needs, so an easy trajectory is settled in a few rounds while a longer one triggers more, which Section 4.9 analyzes.

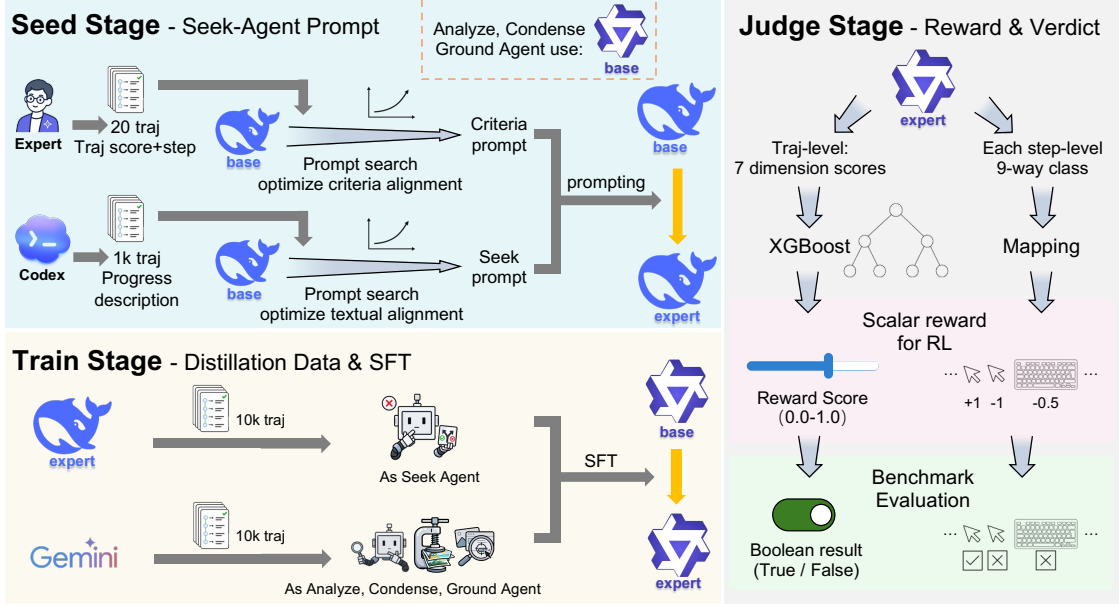


Figure 4 The seed-driven construction of training data, the evaluation process, and how the dense labels are reduced to a trajectory verdict for benchmarking and per-step rewards for RL training.

3.2 Dense Supervision via Seed-Calibrated Distillation

To train the single 9B backbone that the four agents share, we build supervised data for each of the four roles through the pipeline that Figure 4 summarizes.

Dense label design. A single true/false label is coarse. It cannot tell a strong trajectory from a weak one, nor separate the distinct axes on which a trajectory succeeds or fails, such as the fraction of the task completed or the confidence of the judgment. We therefore score each trajectory on seven dimensions, each on a 0–100 scale. At the step level we likewise avoid a bare true/false mark, assigning each step one of nine classes instead. The criteria prompt that produces these dense labels is detailed in Appendix A.

Seed-driven data construction. We adopt a seed-driven construction that reconciles high human alignment with large-scale coverage. In the **Seed Stage**, our goal is to make the closed-source DeepSeek perform better as the Seek agent. This needs the two prompts, the criteria prompt that produces the dense labels, and the seek prompt that decides what to ask the Analyze agent and when to stop. (1) For the criteria prompt, a human first fully labels a small set of trajectories. Each label carries the seven trajectory-level 0–100 scores and a nine-way class for every step. A small prompt search then tunes the criteria prompt so DeepSeek, run over a Seek-agent context, matches these human labels. (2) For the seek prompt, Codex with GPT-5.5 annotates a larger set with a progress description of each trajectory. A larger prompt search then narrows the gap between DeepSeek’s reading of a trajectory and Codex’s textual descriptions. Both searches run through Claude Code, which reads each case’s failures and revises the prompt. In the **Train Stage**, we collect the data that trains the four agents. We use the criteria and seek prompts from the Seed Stage and run the full SEEKJUDGE pipeline, with DeepSeek V3.2 for the text roles and Gemini 3.0 Flash Preview for the vision calls. We record every agent call as distillation data, then train all four roles into one unified model.

Judge Stage: From dimensions to a scalar reward. The framework emits seven overall dimension scores for the trajectory and a nine-way class label for each step, and we reduce both to the scalars that training and benchmarking consume. For the trajectory verdict, we fit a small gradient-boosted regressor on the human-anchored subset that maps the seven overall scores to a trajectory score as training signal used in RL, then fit a threshold on this score to produce a binary verdict aligned with the human label. For the steps, we map each of the nine classes to a preset constant fixed by a human rubric. In RL these per-step constants are

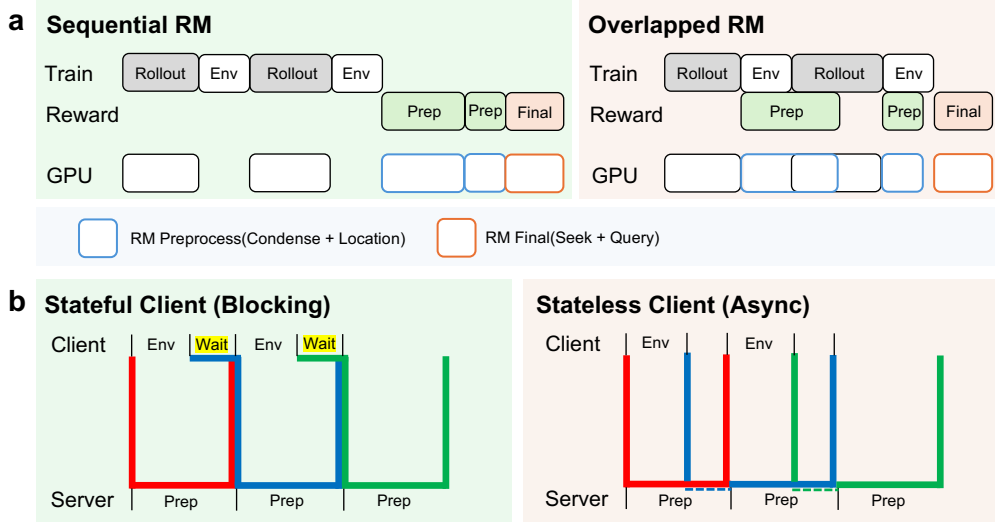


Figure 5 Rollout-overlapped, zero-client-state reward server. **(a)** A sequential reward model starts only after the trajectory ends and stalls training (left), while SEEKJUDGE schedules the preprocessing into the idle GPU windows during the rollout, leaving only the short final stage on the critical path (right). **(b)** A stateful client blocks between environment steps to keep requests ordered (left), while our stateless client streams screenshots and actions out of order and the server reorders the stream and runs the preprocessing asynchronously (right).

aggregated into the trajectory-level reward by the rule of Appendix D, while the framework still exposes the full per-step labels as its step-level output. When step-level benchmarking instead reads the steps as a binary correctness judgment, we collapse the nine classes by treating the three harmful ones as erroneous and the remaining six as correct.

Training data. We collect a large-scale and diverse training set sampled by task difficulty across three platforms, web, OS and Android. We report its composition in Section 4. The training data has zero overlap with any reward benchmark we tested or the OSWorld goal.

3.3 Rollout-Overlapped, Zero-Client-State Reward Server

Rollout overlap. We overlap judging with the rollout so the reward model adds less latency to training. Existing model-based evaluation starts only after a trajectory ends, so its latency is paid as a stall on top of training. Inside a reward-model framework some operations do not depend on the actions that follow, such as Condense and Ground in our framework or key-information identification in CUAJudge. We schedule these operations in the idle GPU window while the environment executes the action, so they stay hidden behind the rollout. Figure 5 shows the design.

Zero client state. We move the scheduling complexity that rollout overlap introduces from the RL side to the reward server. The RL side keeps no reward-related state and only streams each screenshot and action as it is produced, out of order. The server reorders the stream and schedules the overlapped computation, so supporting the overlap needs no change to the RL-side code, which makes this reward server architecture a general choice for multi-turn RL.

4 Experiments

4.1 CUASTEPBENCH

To evaluate a reward model for computer-use agents, we build CUASTEPBENCH, a human-annotated benchmark of 278 tasks over 177 applications. Every trajectory carries a human verdict, and beyond that

Table 1 CUAStepBENCH compared with existing reward benchmarks. Platform icons denote  Web,  Ubuntu,  Windows,  macOS,  Android, iOS. CUAVerifierBench () only provides a coarse progress description rather than a step-level reward. OS-Critic Bench () labels isolated candidate actions before execution, rather than assigning post-hoc rewards to executed steps.

Benchmark	Coverage	Step reward	Human-labeled
AgentRewardBench (Lù et al., 2025)		×	✓
OmniGUIRewardBench (Li et al., 2026a)	    	×	×
CUARewardBench (Lin et al., 2025)		✓	✓
CUAVerifierBench (Rosset et al., 2026)			✓
OS-Critic Bench (Wu et al., 2026)	    		✓
CUAStepBENCH (ours)	     	✓	✓

Table 2 Reinforcement-learning main results. Each reward model drives RL training of an actor across three task domains, and we report the training reward (Train) and the test success rate (Test), whose reporting protocol and run-to-run standard deviations are detailed in Section 4.2; the Qwen-OS SEEKJUDGE run is pending.

Backbone	Reward	Chrome		Impress		OS	
		Train	Test	Train	Test	Train	Test
UI-TARS 1.5 7B	Rule-based	41.33	12.75	43.00	30.43	55.32	25.56
	CUAJudge	34.15	<u>13.91</u>	26.60	<u>33.62</u>	<u>49.83</u>	<u>26.11</u>
	OSThemis	12.64	10.14	19.59	32.46	20.45	25.00
	SEEKJUDGE	<u>37.40</u>	16.23	<u>30.72</u>	36.81	49.11	28.89
Qwen3VL-8B	Rule-based	54.89	<u>14.49</u>	43.84	49.28	69.09	32.22
	CUAJudge	47.56	14.20	27.35	40.58	66.97	8.89
	OSThemis	42.57	11.01	20.58	40.87	65.14	25.00
	SEEKJUDGE	<u>51.59</u>	15.36	<u>33.84</u>	<u>48.41</u>	—	—

verdict each step receives a label. Table 1 places CUAStepBENCH among existing reward benchmarks. We further build CUAStepBENCH-LONG, a long-horizon extension of CUAStepBENCH that holds 18 trajectories with a mean length of 272 steps and dense human annotation, which probes how judging scales to long trajectories in Section 4.8.

Mining hard cases for a discriminative benchmark. Judging is hardest when the actor’s behavior diverges from what actually happened on screen, and exactly these cases bound how far a reward can push an actor. To concentrate on them, we run our closed-source data-generation judge (Section 3.2) over the candidate pool and keep the tasks it scores as borderline between success and failure. We balance the retained tasks across step counts and applications, then have a human label each one. The filter turns only on our own judge’s uncertainty, so the benchmark stays clean and its absolute F1 sits below other reward benchmarks purely because the tasks are harder.¹

4.2 Training and Evaluation details

We train a separate policy for every application rather than a single model shared across domains, so each run measures how far a small actor can be pushed within one setting. Every run uses GRPO on 8×A100 GPUs for 75 training steps. We evaluate on the held-out test set every 15 steps, and to damp the run-to-run noise of any single checkpoint we report each policy’s test success as the mean over all test evaluations taken during training rather than the number at a single step. The training success rate is reported as an EMA over training steps. From repeated runs on the UI-TARS backbone, the run-to-run standard deviation is

¹Low numbers elsewhere can have a different origin. Part of the low metrics on AgentRewardBench has been attributed to annotation issues (<https://github.com/McGill-NLP/agent-reward-bench/issues/9>).

Table 3 Offline evaluation on CUASTEPBENCH, reporting trajectory- and step-level Accuracy, Precision, Recall and F1 in %. Step-level metrics are computed with respect to error steps. [†] marks frameworks without native step-level judgments, whose step labels are instead obtained through the step extraction procedure of SEEKJUDGE on the same base model.

Framework	Model	Trajectory-level				Step-level			
		Acc	Prec	Recall	F1	Acc	Prec	Recall	F1
<i>Closed-source models</i>									
Codex	GPT-5.5	79.1	87.1	70.1	77.7	–	–	–	–
CUAJudge	GPT-5-mini	71.1	78.6	61.1	68.8	90.4	49.7	30.8	38.0 [†]
<i>Open-source models</i>									
CUAJudge	Qwen3VL-8B	67.9	72.0	62.5	66.9	90.9	67.7	12.7	21.3 [†]
OSThemis	Qwen3VL-8B	63.8	71.1	51.2	59.5	83.6	17.0	14.3	15.6 [†]
SEEKJUDGE	Qwen3VL-8B	61.7	58.7	89.4	<u>70.8</u>	89.4	41.4	20.4	<u>27.3</u>
SEEKJUDGE	SeekJudge-9B	73.1	73.0	76.1	74.5	89.6	44.5	33.3	38.1

about 2.0% for the test success rate and about 2.7% for the training reward. The remaining RL training hyperparameters and the full configuration are in Appendix D.

A calibration turns the seven dimension scores into a scalar, and unless noted otherwise the whole paper uses only two parameters. The first serves offline evaluation. We score every offline benchmark with a single shared calibration, applying the same fit to a trained specialist and to an untrained base model alike, obtained by the procedure of Section 3.2. The second serves RL. We fit this calibration on base-model rollouts and hold out the dimensions that a goal fixes on its own rather than ones a trajectory earns, such as goal infeasibility. GRPO scores rollouts relative to a group that shares one goal, so a goal-level dimension takes the same value across the whole group and cannot separate a stronger trajectory from a weaker one; keeping it only adds an offset that the group-relative advantage cancels. This exclusion follows from the group structure alone and is independent of the rule verifier used at test time, so it introduces no coupling between the training reward and the test metric. Appendix B details our leakage control and shows that neither fit overfits its calibration set.

4.3 Reinforcement Learning Main Results

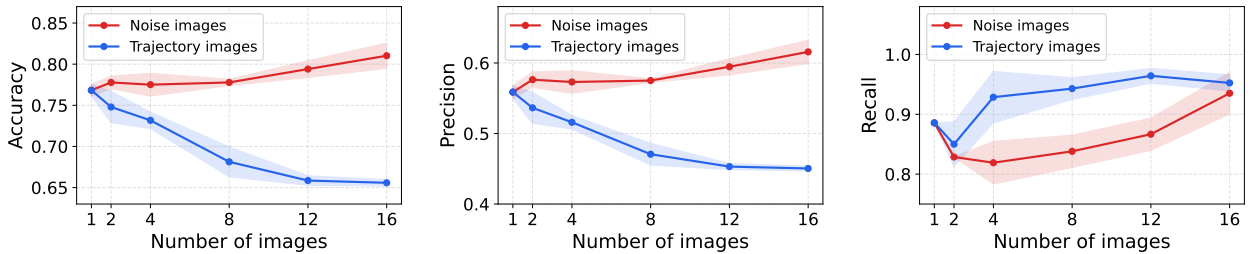
Table 2 reports the test success rate of policies trained under each reward model. The results show that **(1)** SEEKJUDGE matches or exceeds native rule-based supervision on test success, while rule-based takes the highest training reward because its training signal is exactly the reward metric, so its training lead reflects overfitting rather than transfer. **(2)** Across both actor backbones and all three environments, SEEKJUDGE beats the other two model-based frameworks on both training and test reward, including CUAJudge despite its reliance on the closed-source GPT-5-mini.

4.4 Offline Reward Benchmark Evaluation

We evaluate SEEKJUDGE as a static judge on three reward benchmarks before placing it inside the RL loop. CUASTEPBENCH scores both the trajectory verdict and the step labels, while AgentRewardBench and OmniGUIRewardBench score the trajectory verdict alone. The results in Tables 3 and 4 show that **(1)** under a matched Qwen3VL-8B backbone, SEEKJUDGE beats OSThemis on trajectory F1 across all three benchmarks, which attributes the gain to the judging framework rather than the model. **(2)** Training the SeekJudge-9B specialist sharpens the trajectory verdict and the step-level reading together, lifting trajectory F1 over the Qwen3VL-8B base by 3.7 to 12.2 points and step-level F1 from 27.3 to 38.1 on CUASTEPBENCH. **(3)** SeekJudge-9B surpasses the closed-source CUAJudge and WebJudge despite their GPT-5-mini and o4-mini calls, and it is the first judge to substantially clear the Rule baseline on AgentRewardBench.

Table 4 Offline evaluation on AgentRewardBench and OmniGUIRewardBench, reporting trajectory-level result in %. [‡] marks results quoted from their source papers

Framework	Model	AgentRewardBench				OmniGUIRewardBench			
		Acc	Prec	Recall	F1	Acc	Prec	Recall	F1
Rule [‡]	–	–	83.8	55.9	67.1	–	–	–	–
WebJudge [‡]	o4-mini	–	82.0	47.8	60.4	–	–	–	–
WebJudge [‡]	Qwen + o4-mini	–	75.7	58.0	65.6	–	–	–	–
World-State-Model [‡]	Specialized-7B	–	71.2	72.2	<u>71.7</u>	–	–	–	–
CUAJudge	GPT-5-mini	82.9	75.2	53.6	62.6	84.9	90.6	78.0	<u>83.8</u>
OSThemis	Qwen3VL-8B	80.4	73.1	42.4	53.6	77.3	88.4	63.1	73.7
SEEKJUDGE	Qwen3VL-8B	76.9	55.2	72.1	62.5	77.3	74.0	84.2	78.8
SEEKJUDGE	SeekJudge-9B	87.7	82.4	68.4	74.7	85.3	88.8	80.7	84.5

**Figure 6** Accuracy, precision, and recall as the number of images in a single judging forward pass grows, with the decisive screenshot always present. Padding uses other screenshots from the same trajectory (*blue*) or information-free noise images of equal token budget (*red*).

4.5 Multi-Image Information Noise

This experiment asks whether reading a detail from an image degrades when many images share one forward pass. To isolate this extraction from localization, we always place the decisive screenshot in the input, keeping the 123 of 278 CUASTEPBENCH cases whose completion can be settled from a single screenshot. Holding this screenshot fixed, we pad the input with other screenshots from the same trajectory in their original order (blue), or with the same number of pure-mosaic images that occupy an identical token budget but carry no readable content (red). Figure 6 reports the result.

The results show that **(1)** competing content rather than context length drives the drop. Blue and red carry the same context length, so their gap isolates the effect of trajectory content from that of length alone. Blue F_1 falls monotonically from 0.68 to 0.61 as images accumulate, even though the decisive image is always present, while red instead rises, so the added trajectory detail dilutes the verdict and length alone never hurts. The rise of red matches Jang et al. (2025), where extra uninformative tokens widen the model’s computation and add parallel scratch space. **(2)** More images push the judge toward accepting the trajectory. Splitting blue F_1 into its terms, recall barely moves while precision collapses from 0.56 to 0.45, since the judge reads the accumulating plausible-looking details as evidence of success and the one decisive screenshot is drowned out.

4.6 Inference Cost

A reward model is queried once per training step, so its per-case cost decides whether reinforcement learning stays affordable. We price every judge on the same 43 cases under one cost model, converting both the self-deployed open-source models and the closed-source API calls to a common dollar cost per token. Appendix E gives the token prices, the closed-source rates, and the prefix-cache rule we apply uniformly to every method. Figure 7 shows that SEEKJUDGE judges at the lowest cost at every image count, and its lead widens with trajectory length. OSThemis and CUAJudge pay extra gpt-5-mini calls on top of their open-source agents, and the agentic Codex baseline costs two orders of magnitude more.

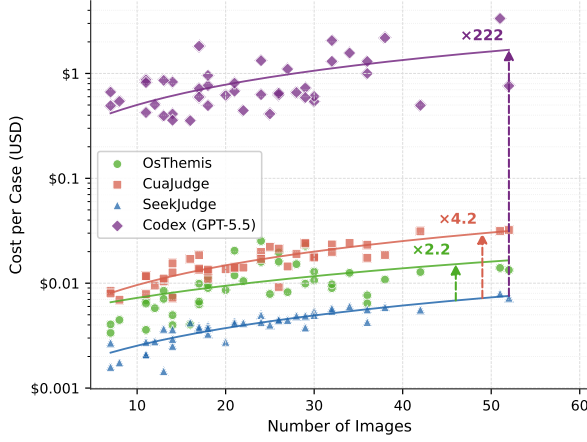


Figure 7 Per-case judging cost against the number of images in a trajectory, where each marker is one case and each line is a linear fit; the cost axis is logarithmic, so the fitted lines appear curved.

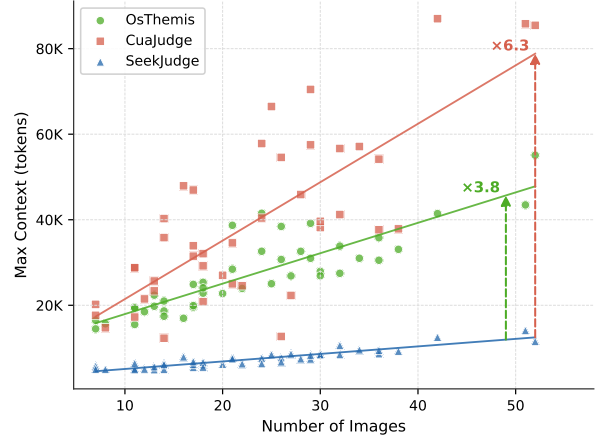


Figure 8 Peak per-call context size against trajectory length, where each marker is one case and each line a linear fit.

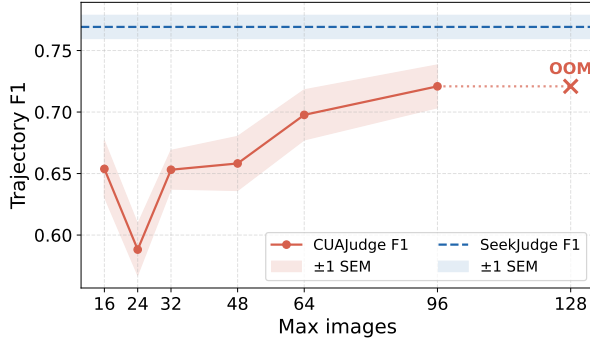


Figure 9 Trajectory F1 on CUAStepBENCH-LONG as CUAJudge's max-image cap grows, with SEEKJUDGE as a cap-free baseline on the same Qwen3VL-8B backbone. CUAJudge uses 4× the deploy resource of SEEKJUDGE. Bands are ± 1 SEM over 8 runs per cap (16 for SEEKJUDGE).

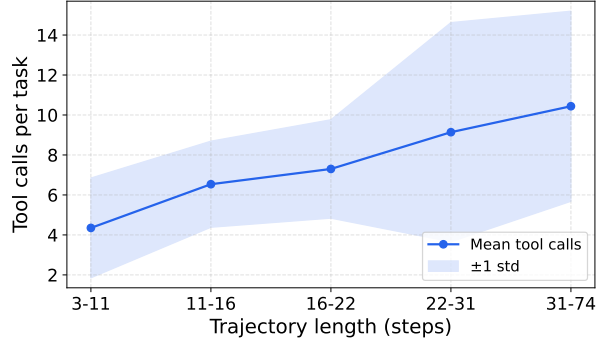


Figure 10 Mean number of tool calls per case against trajectory length, with a ± 1 standard deviation band.

4.7 Context Scaling

A reward model that keeps a small per-call context can judge longer trajectories and is cheaper to train, since the context a judge holds at its peak sets both the longest trajectory it can accept within a fixed window and the activation memory it consumes during training. Figure 8 plots, for each case, the token count of the single largest request a judge issues against the number of images in the trajectory. The peak context of SEEKJUDGE stays nearly flat as trajectories grow, rising to about 12K tokens at 52 images while OSThemis reaches roughly 48K and CUAJudge roughly 80K, a 4–6× gap that widens with length. The same small context lowers the deployment memory of SEEKJUDGE on long trajectories.

4.8 Image Budget on Long Trajectories

Section 4.5 isolates the extraction stage by always keeping the decisive screenshot in the input, so this experiment probes the complementary regime where localization becomes the bottleneck. We run CUAJudge on CUAStepBENCH-LONG under a growing max-image cap. Figure 9 reports the result.

The results show that the single-forward judge loses on both sides. **(1)** With a small cap the decisive screenshot

Table 5 RL Reward-granularity ablation on the UI-TARS 1.5 7B Impress domain. We vary the granularity of the SEEKJUDGE reward and compare against rule-based supervision, reporting the training reward (Train), test success rate (Test), both in %, and the gradient norm (Grad Norm, EMA 0.95). The finest-grained reward, a continuous score augmented with the per-step scores, attains the best test success.

Reward	Granularity	Train	Test	Grad Norm
Rule-based	Boolean	43.00	30.43	1.766
SEEKJUDGE	Boolean	21.08	35.94	2.936
SEEKJUDGE	Continuous + step-level	30.72	36.81	3.446

Table 6 Framework ablation on CUASTEPBENCH, reporting trajectory-level (Traj) and step-level (Step) F1 in %. The first row of each block is the full SEEKJUDGE, and DS + Gemini denotes DeepSeek-V4-Pro as SeekAgent paired with Gemini-3.0-Flash as AnalyzeAgent. For Ground, $\circ$ keeps the raw action without pixel grounding and $\times$ removes action information.

Model	Framework component			F1 (%)	
	Condense	Ground	Seek-Analyze	Traj	Step
Qwen3VL-8B	✓	✓	✓	70.8	27.3
	✗	✓	✓	66.7	26.8
	✓	$\circ$	✓	71.5	26.0
	✓	✗	✓	71.4	20.8
	✓	✓	✗	72.0	25.5
Closed-source (DS + Gemini)	✓	✓	✓	73.6	44.0
	✓	✓	✗	72.2	42.4

is often not included at all. F1 rises with the cap from 0.65 at 16 images to 0.72 at 96, the opposite direction of Figure 6, whose guaranteed decisive frame leaves only the extraction-side dilution; here localization coverage dominates, so adding frames helps. **(2)** With a large cap the judge can still misread the decisive screenshot inside the diluted context, so even 96 images reach only 0.72, below the 0.77 of SEEKJUDGE.

4.9 Adaptive Compute

SEEKJUDGE spends compute in proportion to how much evidence a case demands rather than running a fixed pipeline on every trajectory. Figure 10 groups trajectories into five equal-size bins by step count and reports the mean number of tool calls the Seek controller issues in each. Tool calls grow slowly with trajectory length, since the controller re-examines only the steps whose evidence matters for the verdict rather than inspecting every screenshot.

4.10 Reward Granularity for RL

Table 5 ablates the granularity of the SEEKJUDGE reward on the UI-TARS Impress domain. A continuous score augmented with the per-step scores gives the best test success, ahead of a Boolean SEEKJUDGE reward and of rule-based supervision. Both the continuous score and the per-step term supply a denser training signal than the binary 0/1 rule, giving the policy a graded trajectory reward rather than a single pass-or-fail bit at the end. Appendix D details how the per-step scores enter the trajectory reward.

4.11 Framework Ablation

We ablate the three components of SEEKJUDGE on CUASTEPBENCH in Table 6, swapping one component at inference on the Qwen3VL-8B base and on a strong closed-source pair. We do not sweep the trained SeekJudge-9B, whose distillation data follows the full Condense-Ground-Seek-Analyze pipeline, so dropping a stage would push the specialist out of distribution and measure that shift rather than the stage’s value.

The results show that **(1)** Condense and Ground are both indispensable, costing 4.1 points of trajectory F1 and 6.5 points of step F1 when removed. **(2)** The payoff of the Seek–Analyze loop scales with the backbone. It lifts the strong closed-source pair on both levels but not the weaker 8B base, whose low-quality extraction queries motivate distilling the Seek agent from a stronger teacher (Section 3.2). On the same benchmark the trained SeekJudge-9B (74.5/38.1, Table 3) sits well above the 8B base (70.8/27.3) and level with the closed-source pair (73.6/44.0), placing it in the strong-backbone regime where the loop pays off. This is why the specialist runs the full pipeline.

5 Conclusion

We revisited long-trajectory judging for computer-use agents as the composition of localization and extraction, showing that stacking more images into one forward pass dilutes the decisive evidence even when it is present. SEEKJUDGE realizes this decomposition with four agents on a single 9B backbone, localizing over a condensed text timeline and extracting from one image at a time. It is the first model-based reward to match or surpass native rule-based supervision on downstream test success in online RL, at a small fraction of closed-source cost and with a 4–6 $\times$ smaller per-call context. Together with CUASTEPBENCH and the rollout-overlapped reward server, these results make model-based reward a practical substitute for rules and extend reinforcement learning for computer-use agents to environments that rules cannot instrument.

A Prompt Construction

The judging process uses three prompts, the seek prompt that explores the trajectory and a criteria prompt that scores it, where the criteria prompt itself splits into an overall-trajectory prompt and a step prompt. Keeping exploration and scoring apart holds the exploration context out of scoring and lets the step prompt move to other frameworks unchanged.

The seek prompt drives exploration. At each round the agent either calls a tool to gather more evidence or emits its conclusion. Emitting a conclusion ends the phase, and otherwise the agent keeps exploring until it reaches the round limit, at which point it is forced to stop and conclude.

Once the conclusion is fixed, the criteria prompt produces the detailed evaluation. It splits into two independent scoring prompts, the overall-trajectory prompt that assigns the seven overall dimension scores and the step prompt that assigns the nine-way class label to each step, each under its own system prompt. The step prompt depends only on the trajectory representation rather than on the exploration context, so it attaches to any framework that lacks native step-level judgment, including CUAJudge and OSThemis, and yields the step-level labels we report for those baselines.

An untrained model runs both phases in full at inference, including the SeekJudge Qwen3VL-8B backbone and CUAJudge. For the trained SeekJudge model we fold the second phase into the first during distillation, so the training data has the model emit the detailed evaluation directly at the end of exploration rather than under a separate scoring prompt. This lowers latency and context cost, simplifies training, and eases later deployment.

B Leakage Control in Calibration

Building SEEKJUDGE touches three places where information could leak from an evaluation target back into the fit, the distillation training data, the prompt search, and the score regression. We control each in turn.

Training data. The distillation set that trains SEEKJUDGE shares no task goal with any downstream target. No goal used in an offline benchmark or in an online RL environment appears among the goals we distill on, so the model is never trained on a case it is later scored on.

Prompt search. A prompt search can overfit the set it is scored against. We therefore score the search on a small seed set of about 20 human-annotated trajectories and draw all remaining supervision from teacher-model distillation, without any further human guidance or benchmark ground truth. The search thus tunes the prompt against a fixed handful of anchors rather than against the benchmarks it is later evaluated on.

Score regression. The regressor that maps the seven dimension scores to a trajectory score reads only seven scalar features and carries few fitted parameters, and we fit it under K -fold cross-validation, so its capacity to overfit is small to begin with.

To probe the residual risk we vary which benchmarks the regressor is fit on and read every fit on all three benchmarks, in Table 7. Our reported setting fits jointly on all three. Against it we place three single-benchmark fits, each of which produces a diagonal cell that fits and evaluates on the same benchmark and off-diagonal cells that evaluate on benchmarks absent from the fit. A diagonal cell is the highest-risk reading, since the regressor has seen the target distribution. An off-diagonal cell is a clean transfer to a benchmark the fit never touched, and the joint fit sits between the two.

Two readings of Table 7 bound the risk. First, the diagonal, the highest-risk fit, never leads its column by a meaningful margin. On AgentRewardBench the clean transfer fit on CUASTEPBENCH even edges past it, and on the other two benchmarks it tops the nearest off-diagonal transfer by at most half a point. A regressor that had memorized its training benchmark would show a clear diagonal advantage, and none appears. Second, the off-diagonal transfers, where the evaluation benchmark is absent from the fit, stay within about five points of the diagonal at worst and usually within two, so removing the benchmark from the fit barely moves the

Table 7 Leakage probe for the score-regression calibration, reporting trajectory-level F1 (%) of the trained SeekJudge-9B backbone. Each row is a calibration fit on a different set of benchmarks and then read on all three. Row 1 is the joint fit on all three benchmarks that we use throughout the paper (light yellow, small overfitting risk). Rows 2–4 each fit on a single benchmark. A diagonal cell fits and evaluates on the same benchmark (light red, highest overfitting risk), while an off-diagonal cell evaluates on a benchmark absent from its fit (light green, lowest overfitting risk).

Calibration fit on	Evaluated on (trajectory F1)		
	AgentRewardBench	CUASTEPBENCH	OmniGUIRewardBench
All three (ours)	74.7	74.5	84.5
AgentRewardBench	76.7	69.8	83.8
CUASTEPBENCH	76.8	74.4	85.0
OmniGUIRewardBench	75.7	73.9	85.3

score. The joint fit we use never leads its column by more than 0.1 points, and on AgentRewardBench it is the lowest entry of all, so its exposure to all three benchmarks buys it no inflation over a fit that never saw the benchmark. The same-benchmark overfitting risk is therefore small, which is what the yellow row in Table 7 is meant to convey, and the joint calibration we report throughout the paper is safe to use.

C Human Agreement on the Fine-Grained Step Labels

Our framework assigns every step one of nine fine-grained labels. To check how well these predictions track human judgment, we compare the framework’s label against our own annotation on 254 steps. Table 8 reports the resulting confusion matrix, with rows giving the human label and columns the framework prediction.

We reorder the labels into three blocks and shade them green, yellow, and red. The green block (**milestone**, **correct_decision**, **error_correction**) marks genuine progress, the yellow block (**neutral**, **off_path**, **missed_correction**) marks steps that neither help nor clearly hurt, and the red block (**wrong_action**, **meaningless**, **false_claim**) marks harmful steps. Most of the mass lies on the diagonal, and almost all of the remaining mass stays inside the same color block: 130 of 254 steps land on the exact diagonal, and 159 of 254 fall within the correct block. Disagreement is therefore dominated by fine distinctions inside a block rather than confusion across the progress, neutral, and harmful regimes.

D Reinforcement Learning Training Configuration

This section gives the full training setup summarized in Section 4.2. We optimize every actor with GRPO and train one policy per application across the two backbones UI-TARS-1.5-7B and Qwen3VL-8B, keeping the domains separate so that each number reflects a small model specialized to a single setting. Every run uses a train batch size of 8 with a group size of 8 rollouts per prompt, a learning rate of $1e-6$, and an actor image-history length of 2, and runs for 75 training steps on $8 \times A100$ GPUs in roughly one day.

For test evaluation we run the policy every 15 training steps, repeating the pass three times over a held-out test set at each checkpoint. The value reported in the main results is the mean over all test evaluations collected during training, which averages out the fluctuation of any single checkpoint. Each rollout episode is capped at 15 environment steps during training and 25 during test evaluation, with a maximum prompt length of 13K tokens and a response length of 512 tokens.

How the reward enters GRPO. Every rollout receives a single trajectory-level scalar reward. The framework emits seven overall dimension scores together with a per-step class label for every step. We pass the seven overall scores through the fitted regressor to an outcome score, map each per-step label to its rubric constant, and set the trajectory reward to the outcome score plus a fixed weight w_{step} times the mean of the per-step constants. For a group of G rollouts that share one goal, the GRPO advantage of rollout i is $(r_i - \mu_G)/\sigma_G$,

Table 8 Confusion matrix between the human label (rows) and our framework’s prediction (columns) over 254 annotated steps. Labels are grouped into three blocks shaded green (progress), yellow (neutral), and red (harmful). Diagonal entries are bold. Agreement concentrates on the diagonal and, failing that, within the same block.

Human	Framework prediction									Total
	milestone	correct _ decision	error _ correction	neutral	off_path	missed _ correction	wrong _ action	meaningless	false _ claim	
	progress	neutral			harmful					
milestone	18	4	0	5	0	3	2	0	0	32
correct_decision	2	9	0	2	1	1	1	0	0	16
error_correction	2	0	2	0	1	0	1	0	0	6
neutral	14	21	6	76	5	2	16	3	0	143
off_path	2	2	1	8	15	0	1	0	0	29
missed_correction	0	1	0	3	2	0	1	1	0	8
wrong_action	3	1	0	3	1	0	4	0	0	12
meaningless	0	0	0	0	0	0	0	6	0	6
false_claim	1	0	0	0	0	0	1	0	0	2
Total	42	38	9	97	25	6	27	10	0	254

and this one scalar is shared by every decision step of the rollout. Note that the per-step scores act only as a shaping term on the trajectory scalar and never form a separate per-step advantage. We average the per-step constants instead of summing them, keeping this term on a common scale so a longer rollout earns no extra reward for taking more steps. We leave per-step advantage estimation to future work.

E Inference Cost Model

This section gives the full cost model behind Figure 7 in Section 4.6. Every judge’s per-case cost is the sum of an open-source token bill and a closed-source API bill, measured on the 43 cases shared by all four judges.

Why token billing rather than GPU time. We price the self-deployed models by tokens rather than by measured GPU-seconds. Wall-clock GPU time reflects not only the work a judge asks for but also how well its framework overlaps that work, so a serial framework that issues its requests one after another, as OSThemis does, spends far more GPU time than a framework that batches or pipelines the same token workload, even when the two encode and generate identical numbers of tokens. Charging by GPU time would therefore fold each framework’s concurrency engineering into its reported cost and penalize the serial baselines for an implementation detail rather than for the judging work they actually do. Token billing at a fixed per-token rate strips out this confound and is the metric a hosted serving provider would in any case charge, so it compares the frameworks on the token workload itself and treats every method on equal terms.

Token prices. Open-source agents are billed at \$0.04 per million input tokens and \$0.20 per million output tokens, the public serving rate of a Qwen3.5-9B or Qwen3VL-8B class model.² These prices cover the agents of SEEKJUDGE and the Selector of OSThemis. Closed-source calls are billed at their provider’s published price. CUAJudge and the gpt-5-mini calls inside the baselines are charged at \$0.25 per million input tokens and \$2.00 per million output tokens, the official OpenAI rate, and the agentic Codex baseline is charged at the GPT-5.5 API spend recorded for each run.

Prefix-cache treatment. We apply one uniform rule to every judge. A self-deployed small model serves a narrow, repetitive workload, so its requests within one case share a long common prefix that the KV cache returns at almost no cost. We therefore charge any request whose prefix repeats an earlier request in the same case at the cache-hit rate, and bill unique input and all output in full. For the Seek agent in SEEKJUDGE and

²<https://deepinfra.com/blog/qwen3-5-9b-api-benchmarks>

the Selector in OSThemis, whose input is dominated by one long, reused context, this counts input tokens only for the single longest request of the case and treats the input of every other request as a cache hit at zero cost. The same rule yields no discount for CUAJudge, which issues a single turn per case and so has no repeated prefix to reuse. Output tokens are always charged in full, since the cache does not cover generation.

The cache treatment matches the deployment. We verify this accounting on isolated per-agent logs from our sglang deployment. Within one case, each turn reports a cached-token count equal to the full token count of the preceding request, so the entire prior context is served from the prefix cache and only the marginal new tokens are re-encoded. Summed over the case, the freshly encoded input therefore collapses to the single longest request, which is what the accounting above charges. The reported cost is thus a faithful estimate of the deployed behavior rather than a worst case that re-encodes the shared context on every call.

References

- Pranjal Aggarwal, Graham Neubig, and Sean Welleck. Gym-anything: Turn any software into an agent environment. *arXiv preprint arXiv:2604.06126*, 2026.
- Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.11896.
- Hao Bai, Alexey Taymanov, Tong Zhang, Aviral Kumar, and Spencer Whitehead. Webgym: Scaling training environments for visual web agents with realistic tasks, 2026. URL <https://arxiv.org/abs/2601.02439>.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.06070.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2401.13919.
- Yifei He, Rui Yang, Hao Bai, Tong Zhang, and Han Zhao. Pro-cua: Process-reward optimization for computer use agents, 2026. URL <https://arxiv.org/abs/2605.29119>.
- Yoonna Jang, Kisu Yang, and Isabelle Augenstein. Expanding computation spaces of llms at inference time, 2025. URL <https://arxiv.org/abs/2509.24884>.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2401.13649.
- Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.03679; introduces the AndroidControl dataset.
- Zehao Li, Zhenyu Wu, Yibo Zhao, Bowen Yang, Jingjing Xie, Zhaoyang Liu, Zhoumianze Liu, Kaiming Jin, Jianze Liang, Zonglin Li, Feng Wu, Bowen Zhou, Zun Wang, and Zichen Ding. Omniguirewardbench: A cross-platform benchmark for gui outcome reward models. *arXiv preprint arXiv:2603.19191*, 2026a. Introduced within the OS-Themis paper (arXiv:2603.19191).
- Zehao Li, Zhenyu Wu, Yibo Zhao, Bowen Yang, Jingjing Xie, Zhaoyang Liu, Zhoumianze Liu, Kaiming Jin, Jianze Liang, Zonglin Li, Feng Wu, Bowen Zhou, Zun Wang, and Zichen Ding. Os-themis: A scalable critic framework for generalist gui rewards. *arXiv preprint arXiv:2603.19191*, 2026b. Multi-agent critic with milestone verification; also introduces OmniGUIRewardBench.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.20050.

- Haojia Lin, Xiaoyu Tan, Yulei Qin, Zihan Xu, Yuchen Shi, Zongyi Li, Gang Li, Shaofei Cai, Siqi Cai, Chaoyou Fu, Ke Li, and Xing Sun. Cuarewardbench: A benchmark for evaluating reward models on computer-using agent. *arXiv preprint arXiv:2510.18596*, 2025. Youtu Team, Tencent.
- Xing Han Lü, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. Agentrewardbench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942*, 2025.
- Jiayi Pan, Yichi Zhang, Nicholas Tomlin, Yifei Zhou, Sergey Levine, and Alane Suhr. Autonomous evaluation and refinement of digital agents. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2404.06474.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Xinyue Yang, Jiadai Sun, Yu Yang, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2411.02337.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- Christopher Rawles, Sarah Clinckemallie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2405.14573.
- Corby Rosset, Pratyusha Sharma, Andrew Zhao, Miguel Gonzalez-Fernandez, and Ahmed Awadallah. The art of building verifiers for computer use agents. *arXiv preprint arXiv:2604.06240*, 2026. Microsoft Research; introduces the Universal Verifier and CUAVerifierBench.
- Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pp. 3135–3144, 2017.
- Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. Seagent: Self-evolving computer use agent with autonomous learning from experience. *arXiv preprint arXiv:2508.04700*, 2025.
- Bowen Wang, Dunjie Lu, Junli Wang, Tianyi Bai, Shixuan Liu, Zhipeng Zhang, Haiquan Wang, Hao Hu, Tianbao Xie, Shuai Bai, Dayiheng Liu, Que Shen, Junyang Lin, and Tao Yu. Cua-gym: Scaling verifiable training environments and tasks for computer-use agents. *arXiv preprint arXiv:2605.25624*, 2026.
- Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a. ByteDance Seed.
- Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2312.08935.
- Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, et al. Opencua: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025b.
- Zhenyu Wu, Jingjing Xie, Zehao Li, Bowen Yang, Qiushi Sun, Zhaoyang Liu, Zhoumianze Liu, Yu Qiao, Xiangyu Yue, Zun Wang, and Zichen Ding. Os-oracle: A comprehensive framework for cross-platform gui critic models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 27514–27524, 2026. Introduces OS-Critic Bench.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.07972.
- Tianbao Xie, Mengqi Yuan, Danyang Zhang, Xinzhuang Xiong, Zhennan Shen, Zilong Zhou, Xinyuan Wang, Yanxu Chen, Jiaqi Deng, Junda Chen, Bowen Wang, Haoyuan Wu, Jixuan Chen, Junli Wang, Dunjie Lu, Hao Hu, and Tao Yu. Introducing osworld-verified. *xlang.ai*, July 2025. URL <https://xlang.ai/blog/osworld-verified>.

- Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents. In *Second Conference on Language Modeling (COLM)*, 2025. arXiv:2504.01382; introduces Online-Mind2Web and the WebJudge evaluator.
- Tianci Xue, Zeyi Liao, Tianneng Shi, Zilu Wang, Kai Zhang, Dawn Song, Yu Su, and Huan Sun. Autonomous continual learning of computer-use agents for environment adaptation. *arXiv preprint arXiv:2602.10356*, 2026. Introduces CUAJudge, a WebJudge-based automatic evaluator of CUA trajectories.
- Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, Xuehui Wang, Zhaoyang Liu, Jinguo Zhu, Hao Li, Wenhai Wang, Yu Qiao, Xizhou Zhu, and Jifeng Dai. Zerogui: Automating online gui learning at zero human cost. *arXiv preprint arXiv:2505.23762*, 2025.
- Rui Yang, Qianhui Wu, Yuxi Chen, Hao Bai, Wenlin Yao, Hao Cheng, Baolin Peng, Huan Zhang, Tong Zhang, and Jianfeng Gao. Openwebrl: Demystifying online multi-turn reinforcement learning for visual web agents. *arXiv preprint arXiv:2606.02031*, 2026.
- Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025. Introduces the GUI-Owl foundational agent.
- Mengqi Yuan, Tianbao Xie, Tao Yu, et al. Osworld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026. XLANG Lab.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.05685.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.13854.
- Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, et al. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.